Bináris fájlok kezelése C#-ban

A C#-ban a bináris fájlok kezelése a System.IO névtér eszközeire épül.

Míg a szöveges fájloknál karakterkódolással dolgozunk, a bináris fájlok közvetlenül a nyers bájtokat vagy a típusos adatokat (egész számok, lebegőpontos értékek, bájttömbök) tárolják és olvassák vissza.

A két leggyakoribb megközelítés:

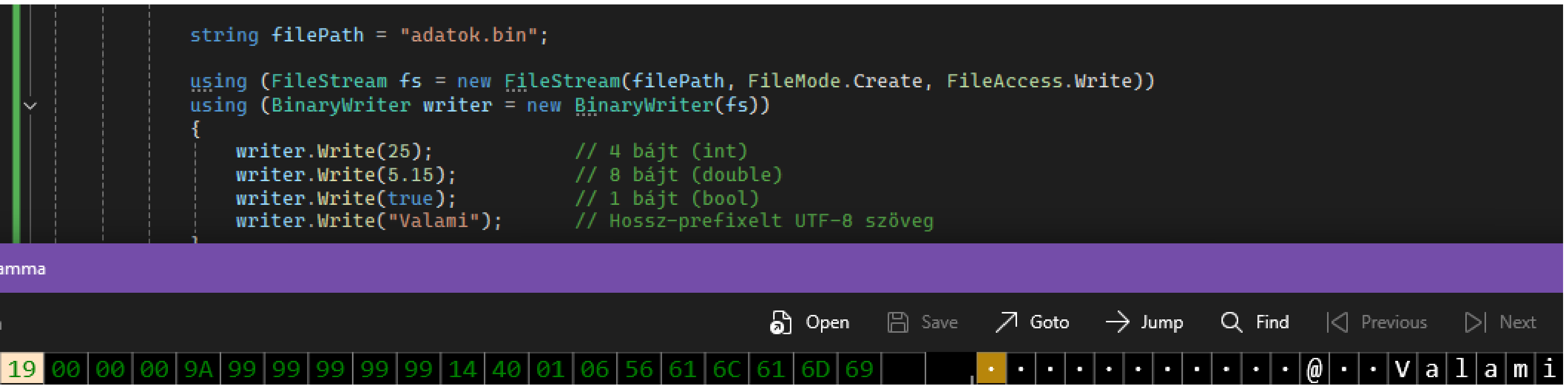
- BinaryWriter és BinaryReader: Típusos primitív adatok (pl. int, double, bool, string) közvetlen írása és olvasása.
- FileStream közvetlen bájtműveletekkel: Nyers bájtok (byte[]) írása, olvasása és tetszőleges pozicionálás (Seek).

1. Típusos adatok kezelése: BinaryWriter és BinaryReader

Ez a legkényelmesebb módszer, ha struktúrált adatokat szeretnénk bináris formában elmenteni.

Írás a fájlba

A BinaryWriter automatikusan a megfelelő bájtsorrendben és méretben írja ki az értékeket

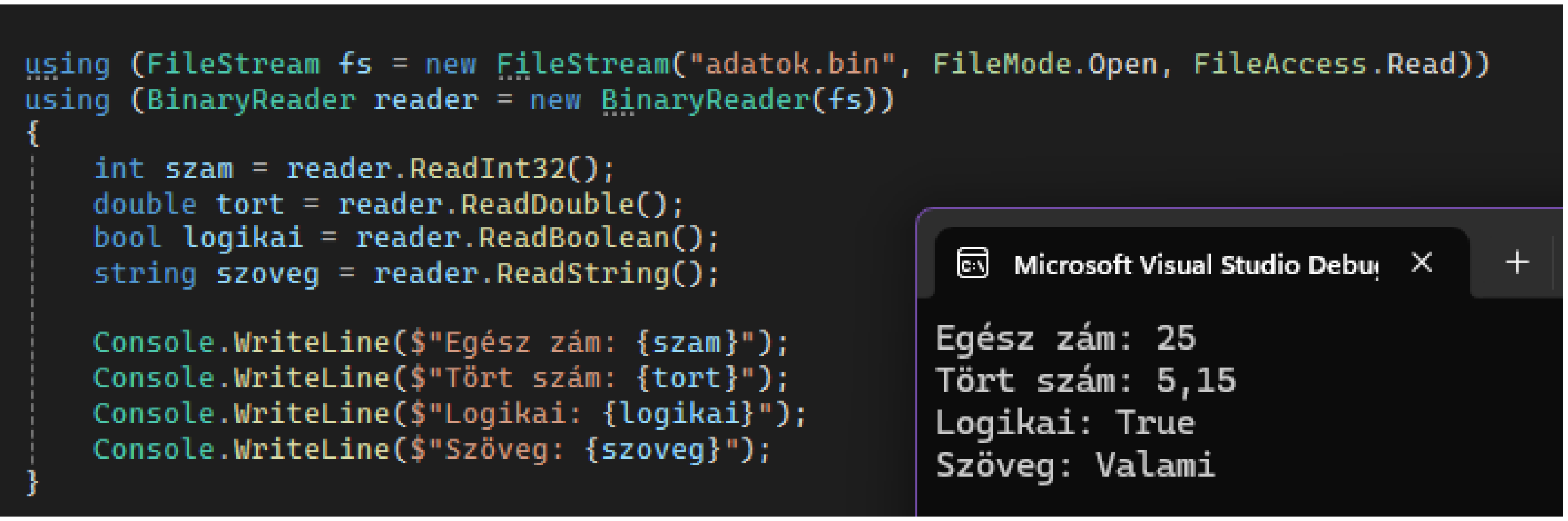


```
string filePath = "adatok.bin";

using (FileStream fs = new FileStream(filePath, FileMode.Create, FileAccess.Write))
using (BinaryWriter writer = new BinaryWriter(fs))
{
    writer.Write(25);              // 4 bájt (int)
    writer.Write(5.15);            // 8 bájt (double)
    writer.Write(true);            // 1 bájt (bool)
    writer.Write("Valami");        // Hossz-prefixelt UTF-8 szöveg
```

```
19 00 00 00 9A 99 99 99 99 99 14 40 01 06 56 61 6C 61 6D 69    · · · · · · · · · · · · @ · · V a l a m i
```

Olvasás a fájlból

A visszaolvasásnál a típusok sorrendjének és típusának pontosan meg kell egyeznie az írásával

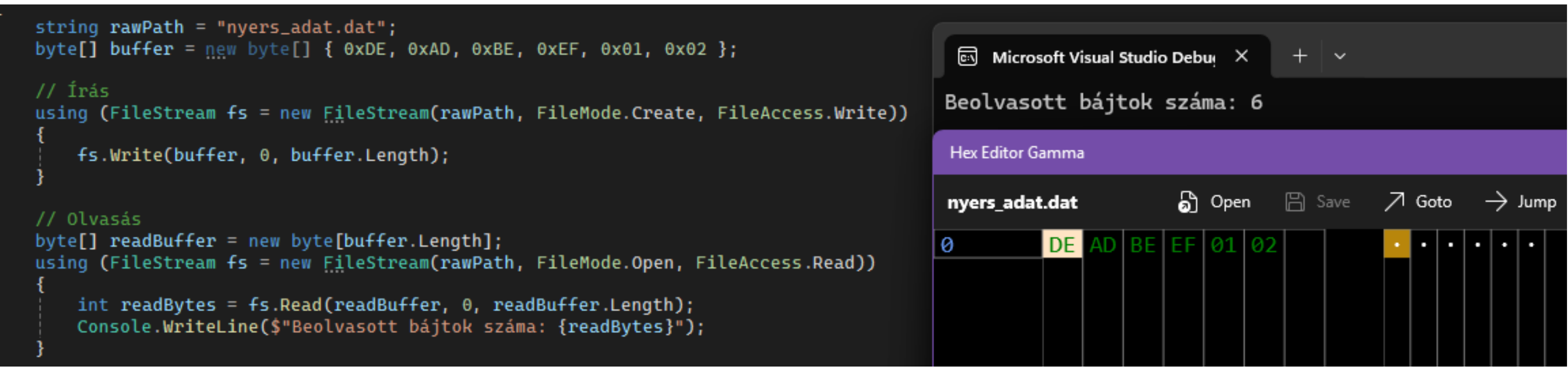


```
using (FileStream fs = new FileStream("adatok.bin", FileMode.Open, FileAccess.Read))
using (BinaryReader reader = new BinaryReader(fs))
{
    int szam = reader.ReadInt32();
    double tort = reader.ReadDouble();
    bool logikai = reader.ReadBoolean();
    string szoveg = reader.ReadString();

    Console.WriteLine($"Egész zám: {szam}");
    Console.WriteLine($"Tört szám: {tort}");
    Console.WriteLine($"Logikai: {logikai}");
    Console.WriteLine($"Szöveg: {szoveg}");
}
```

```
Egész zám: 25
Tört szám: 5,15
Logikai: True
Szöveg: Valami
```

2. Nyers bájtok kezelése: FileStream

Ha képeket, hálózati csomagokat vagy tetszőleges bináris állományokat mozgatunk, közvetlenül bájttömbökkel dolgozhatunk



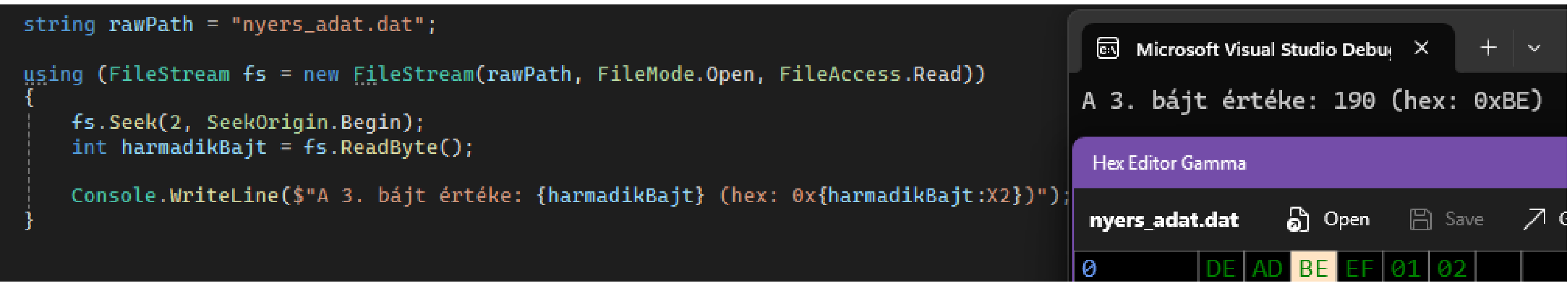
```
string rawPath = "nyers_adat.dat";
byte[] buffer = new byte[] { 0xDE, 0xAD, 0xBE, 0xEF, 0x01, 0x02 };

// Írás
using (FileStream fs = new FileStream(rawPath, FileMode.Create, FileAccess.Write))
{
    fs.Write(buffer, 0, buffer.Length);
}

// Olvasás
byte[] readBuffer = new byte[buffer.Length];
using (FileStream fs = new FileStream(rawPath, FileMode.Open, FileAccess.Read))
{
    int readBytes = fs.Read(readBuffer, 0, readBuffer.Length);
    Console.WriteLine($"Beolvasott bájtok száma: {readBytes}");
}
```

```
Beolvasott bájtok száma: 6
```

```
0   DE AD BE EF 01 02
```

Pozicionálás (Seek)

A FileStream lehetőséget ad arra, hogy a fájl tetszőleges pontjára ugorjunk anélkül, hogy a teljes fájlt végigolvasnánk



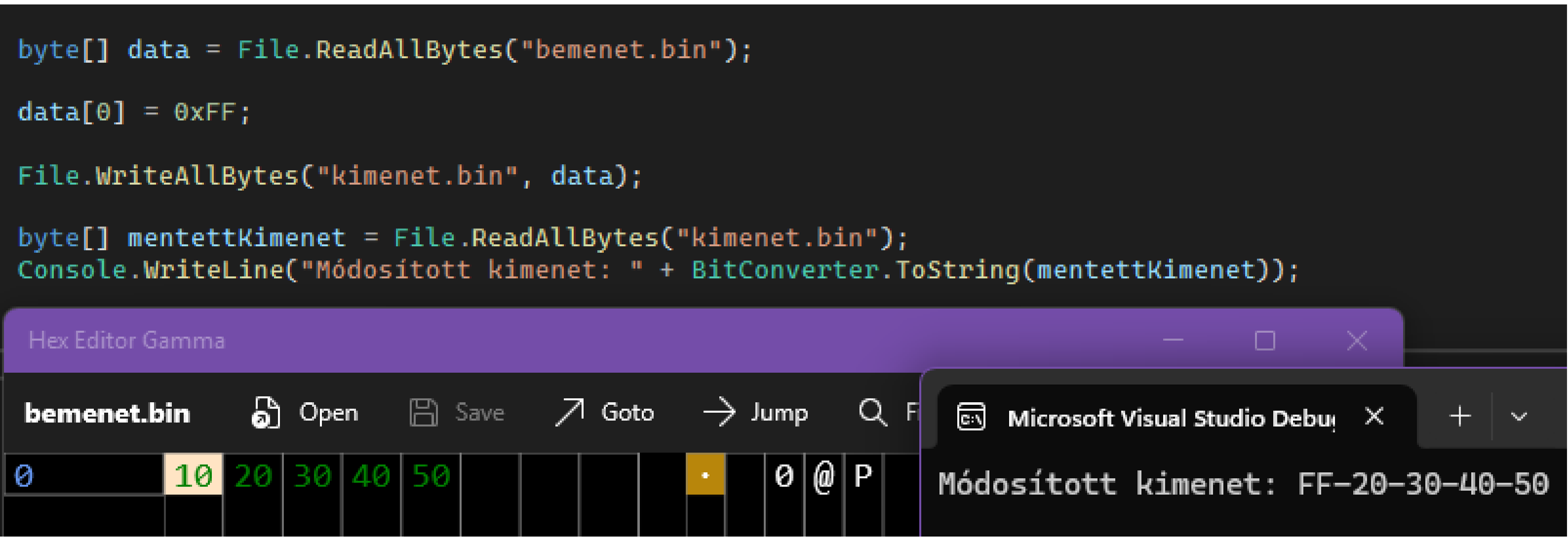
```
string rawPath = "nyers_adat.dat";

using (FileStream fs = new FileStream(rawPath, FileMode.Open, FileAccess.Read))
{
    fs.Seek(2, SeekOrigin.Begin);
    int harmadikBajt = fs.ReadByte();

    Console.WriteLine($"A 3. bájt értéke: {harmadikBajt} (hex: 0x{harmadikBajt:X2})");
}
```

```
A 3. bájt értéke: 190 (hex: 0xBE)
```

```
0   DE AD BE EF 01 02
```

3. Gyorssegéd: File.ReadAllBytes és File.WriteAllBytes

Kisebb fájlok esetén nincs szükség manuális stream-kezelésre



```
byte[] data = File.ReadAllBytes("bemenet.bin");

data[0] = 0xFF;

File.WriteAllBytes("kimenet.bin", data);

byte[] mentettKimenet = File.ReadAllBytes("kimenet.bin");
Console.WriteLine("Módosított kimenet: " + BitConverter.ToString(mentettKimenet));
```

```
0   10 20 30 40 50        .    0 @ P
```

```
Módosított kimenet: FF-20-30-40-50
```

Több száz megabájtos vagy gigabájtos fájloknál kerülendő a File.ReadAllBytes, mert a teljes tartalmat egyszerre a memóriába (RAM-ba) tölti, ami könnyen memóriahiányhoz (OutOfMemoryException) vezethet. Nagy fájloknál mindig darabolva (streamelve) érdemes olvasni.